

Atacar WordPress

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

WordPress es el CMS mas comun en rooms. Su nucleo se actualiza solito, asi que rara vez es el vector. El fallo vive en los plugins y temas de terceros (miles, muchos sin mantener) o en credenciales debiles del panel /wp-admin, desde donde se llega a ejecución editando un tema.

Cada plugin es código de terceros con su propia superficie; una versión vulnerable de un plugin popular es explotable en masa. Y el panel de admin permite editar los .php del tema: si entras, tienes RCE casi directo. wpscan automatiza detectar versión de core, plugins, temas y usuarios.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando whatweb o /wp-login.php delatan WordPress. Encaja tras el fingerprint web, antes de intentar bugs genericos.

Cómo identificarla

- Rutas /wp-content/, /wp-login.php, /wp-json/ o el meta generator con la version.
- wpscan lista un plugin con CVE conocido o una version vieja.
- Usuarios enumerados via /?author=1 o la API REST -> objetivos de spraying.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

wpscan --url \$URL --enumerate vp,u y revisa si algun plugin sale marcado [!] vulnerable.

Evidencia esperada: Version del core, lista de plugins con su version, y usuarios enumerados via /?author= o REST API.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.

4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
wpscan --url $URL --enumerate vp,u --plugins-detection aggressive
```

Objetivo y contexto: Enumera plugins vulnerables (vp) y usuarios (u). La detección agresiva encuentra plugins que la pasiva no ve. Es el primer paso siempre en WordPress.

Resultado esperado: Version del core, plugins con su version y [!] de vulnerabilidad, y la lista de usuarios. Cada plugin vulnerable -> busca su exploit; cada usuario -> candidato a login.

Anatomía de los payloads

La primera prueba es `wpscan --url $URL --enumerate vp,u --plugins-detection aggressive`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Version del core, lista de plugins con su version, y usuarios enumerados via `/?author=` o REST API.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Enumera core, plugins, temas y usuarios con wpscan (usa `--api-token` para ver CVEs).
- Cruza cada plugin/version con exploits conocidos; prioriza los pre-auth.
- Si no hay CVE, ataca el login (usuarios enumerados + passwords) con cuidado de bloqueos.
- Con acceso admin, edita un .php del tema para lograr ejecución y salta a shell (`1fi-a-rce` comparte la idea: escribir código que el servidor ejecuta) y luego `reverse-vs-bind`.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- wpscan no detecta ningún plugin con CVE y el core está en la última versión.
- No hay usuarios enumerables ni login accesible para spraying.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

El core suele estar parcheado; el agujero casi siempre esta en un plugin o tema vulnerable, o en el panel de admin. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (wordpress)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.